

Delta Absorption Scanner[MarkitTick

// This work is licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

// <https://creativecommons.org/licenses/by-nc-sa/4.0/>

// © MarkitTick

//@version=6

indicator("Delta Absorption Scanner[MarkitTick]", overlay=true, max_labels_count=500)

//

===== //

// Inputs & Settings

//

===== //

grp_vis = "Visuals & Colors"

colorPos = input.color(#00E676, "Positive Delta Color", group=grp_vis)

colorNeg = input.color(#FF5252, "Negative Delta Color", group=grp_vis)

colorSpread = input.color(#2a2e39, "Neutral Label Color", group=grp_vis)

colorText = input×color(color×white, "Text Color", group=grp_vis)

lblSize = input×string(size×small, "Label Size", options=[size.auto, size.tiny, size.small, size.normal, size.large], group=grp_vis)

grp_filt = "Filters & Display Options (Chart Labels)"

maxLabels = input.int(10, "Max Labels on Chart", minval=1, maxval=500, group=grp_filt)

minDelta = input.float(10.0, "Minimum Delta % to Show", step=5.0, group=grp_filt)

showSpread = input.bool(true, "Show Spread (S)", group=grp_filt)

showDelta = input.bool(true, "Show Delta % (D)", group=grp_filt)

showCVD = input.bool(true, "Show Session CVD", group=grp_filt)

showDiv = input.bool(true, "Show Divergences", group=grp_filt)

showAbs = input.bool(true, "Show Absorption", group=grp_filt)

showSweep = input.bool(true, "Show Liquidity Sweeps", group=grp_filt)

showFVG = input.bool(true, "Show FVG Imbalances", group=grp_filt)

grp_params = "Algorithmic Parameters"

divLookback = input.int(14, "Divergence Lookback", minval=5, group=grp_params)

absDeltaLimit = input.float(60.0, "Absorption Min Delta %", group=grp_params)

sweepLookback = input.int(20, "Sweep Lookback Period", group=grp_params)

```

grp_risk = "Risk Management & Webhooks (JSON)"
tpPercent = input.float(1.5, "Take Profit (%)", step=0.1, group=grp_risk)
slPercent = input.float(0.5, "Stop Loss (%)", step=0.1, group=grp_risk)
enableAlerts = input.bool(false, "Enable JSON Alerts", group=grp_risk)

grp_tf = "Lower Timeframe Data"
useCustomTF = input.bool(false, "Use Custom LTF", group=grp_tf)
customTF = input.timeframe("1", "Custom Timeframe", group=grp_tf)

grp_dash = "HTF Dashboard Analytics"
showDash = input.bool(true, "Show HTF Dashboard", group=grp_dash)
dashBg = input×color(#1e222d, "Dashboard Background", group=grp_dash)
dashBorder = input×color(#434651, "Dashboard Border", group=grp_dash)
dashText = input×color(color×white, "Dashboard Text", group=grp_dash)

showHtf1 = input.bool(true, "Enable HTF 1", group=grp_dash)
htf1 = input.timeframe("15", "Higher Timeframe 1", group=grp_dash)

showHtf2 = input.bool(true, "Enable HTF 2", group=grp_dash)
htf2 = input.timeframe("60", "Higher Timeframe 2", group=grp_dash)

showHtf3 = input.bool(false, "Enable HTF 3", group=grp_dash)
htf3 = input.timeframe("240", "Higher Timeframe 3", group=grp_dash)

showHtf4 = input.bool(false, "Enable HTF 4", group=grp_dash)
htf4 = input.timeframe("1D", "Higher Timeframe 4", group=grp_dash)

if barstate.islast and (syminfo.type == "renko" or syminfo.type == "heikinashi" or syminfo.type ==
"linebreak" or syminfo.type == "kagi" or syminfo.type == "pnf")
    runtime.error("Warning [MarkitTick]: Non-standard charts detected. HTF data and Webhooks
must use standard candlesticks.")

var label[] activeLabels = array.new_label()

//

```

```

===== //
// Core Methods & Calculations (Chart TF)
//
===== //
upAndDownVolume() =>
    float posVol = 0.0
    float negVol = 0.0
    switch
        close > open    => posVol += volume
        close < open    => negVol += volume
        close >= close[1] => posVol += volume
        close < close[1] => negVol += volume
    [posVol, negVol]

string lowerTimeframe = switch
    useCustomTF      => customTF
    timeframe.isseconds => "1S"
    timeframe.isintraday => "1"
    timeframe.isdaily  => "5"
    => "60"

[upVolumeArray, downVolumeArray] = request.security_lower_tf(syminfo.tickerid, lowerTimeframe,
upAndDownVolume())

float safeUp = na(array.sum(upVolumeArray)) ? 0.0 : array.sum(upVolumeArray)
float safeDn = na(array.sum(downVolumeArray)) ? 0.0 : array.sum(downVolumeArray)
float totalVolume = safeUp + safeDn
float netVolume = safeUp - safeDn

float delta = totalVolume > 0.0 ? (netVolume / totalVolume) * 100.0 : na
float spread = high - low

var float sessionCVD = 0.0
if timeframe.change("D")
    sessionCVD := 0.0
sessionCVD += netVolume

//

```

```

===== //
// Algorithmic Global Data Preparation
//
===== //
float hh_div = ta.highest(high[1], divLookback)
float ll_div = ta.lowest(low[1], divLookback)
float hh_delta = taxhighest(delta[1], divLookback)
float ll_delta = taxlowest(delta[1], divLookback)

float avgSpread = ta.sma(spread, 20)

float hh_sweep = ta.highest(high[1], sweepLookback)
float ll_sweep = ta.lowest(low[1], sweepLookback)

//
===== //
// Algorithmic Detection Logic (Chart TF)
//
===== //
bool isPriceHH = high >= hh_div
bool isDeltaLH = delta < hh_delta
bool bearDiv = isPriceHH and isDeltaLH and not na(delta)

bool isPriceLL = low <= ll_div
bool isDeltaHL = delta > ll_delta
bool bullDiv = isPriceLL and isDeltaHL and not na(delta)

bool bullAbsorb = delta < -absDeltaLimit and spread < (avgSpread * 0.5) and close > low + (spread
* 0.5)
bool bearAbsorb = delta > absDeltaLimit and spread < (avgSpread * 0.5) and close < high - (spread
* 0.5)

bool sweepHigh = high > hh_sweep and close < hh_sweep
bool sweepLow = low < ll_sweep and close > ll_sweep

bool bullFVG = low > high[2] and close > open and delta > 20.0
bool bearFVG = high < low[2] and close < open and delta < -20.0

```

```

//
===== //
// HTF Security Architecture
//
===== //
calc_htf_data(int sweepLb) =>
    float _O = open[1]
    float _H = high[1]
    float _L = low[1]
    float _C = close[1]

    float _spread = _H - _L
    bool _isBull = _C > _O

    int _fvg = (_L > high[3] and _isBull) ? 1 : (_H < low[3] and not _isBull) ? -1 : 0

    float _hh_sweep = ta.highest(high[2], sweepLb)
    float _ll_sweep = ta.lowest(low[2], sweepLb)

    bool _sweepH = _H > _hh_sweep and _C < _hh_sweep
    bool _sweepL = _L < _ll_sweep and _C > _ll_sweep
    int _sweep = _sweepL ? 1 : (_sweepH ? -1 : 0)

    [_spread, _isBull, _fvg, _sweep]

[s1, b1, f1, sw1] = request.security(syminfo.tickerid, htf1, calc_htf_data(sweepLookback),
lookahead=barmerge.lookahead_on)
[s2, b2, f2, sw2] = request.security(syminfo.tickerid, htf2, calc_htf_data(sweepLookback),
lookahead=barmerge.lookahead_on)
[s3, b3, f3, sw3] = request.security(syminfo.tickerid, htf3, calc_htf_data(sweepLookback),
lookahead=barmerge.lookahead_on)
[s4, b4, f4, sw4] = request.security(syminfo.tickerid, htf4, calc_htf_data(sweepLookback),
lookahead=barmerge.lookahead_on)

//
===== //
// HTF Volume Accumulator Engine
//

```

```

===== //
track_htf_delta(string htf_res) =>
    var float accumNet = 0.0
    var float accumTot = 0.0
    var float closedDelta = 0.0

    bool isNewHtf = ta.change(time(htf_res)) != 0

    if isNewHtf
        closedDelta := accumTot > 0.0 ? (accumNet / accumTot) * 100.0 : 0.0
        accumNet := 0.0
        accumTot := 0.0

    accumNet += netVolume
    accumTot += totalVolume
    closedDelta

float d1 = track_htf_delta(htf1)
float d2 = track_htf_delta(htf2)
float d3 = track_htf_delta(htf3)
float d4 = track_htf_delta(htf4)

//
===== //
// Webhook / Alert Engine
//
===== //
bool isBullSignal = bullDiv or bullAbsorb or sweepLow or bullFVG
bool isBearSignal = bearDiv or bearAbsorb or sweepHigh or bearFVG

if enableAlerts and barstate.isconfirmed
    if isBullSignal or isBearSignal
        string action = isBullSignal ? "buy" : "sell"
        float entryPrice = close
        float tpPrice = isBullSignal ? entryPrice * (1.0 + tpPercent / 100.0) : entryPrice * (1.0 -
tpPercent / 100.0)
        float slPrice = isBullSignal ? entryPrice * (1.0 - slPercent / 100.0) : entryPrice * (1.0 + slPercent /
100.0)

```

```

string jsonPayload = '{"action": "' + action + '", ' +
    "'ticker": "' + syminfo.ticker + '", ' +
    "'entry": "' + str.tostring(entryPrice, format.mintick) + '", ' +
    "'take_profit": "' + str.tostring(tpPrice, format.mintick) + '", ' +
    "'stop_loss": "' + str.tostring(slPrice, format.mintick) + '"}'

alert(jsonPayload, alert.freq_once_per_bar_close)

//
===== //
// UI Engine: Dynamic Label Rendering & Memory Management
//
===== //
bool meetsDeltaFilter = not na(delta) and math.abs(delta) >= minDelta

if meetsDeltaFilter
    string labelText = ""

    if showSpread
        labelText += "S: " + str.tostring(spread, "#.###")
    if showDelta
        labelText += (labelText == "" ? "" : "\n") + "D: " + str.tostring(delta, "#.###") + "%"
    if showCVD
        labelText += (labelText == "" ? "" : "\n") + "CVD: " + str.tostring(sessionCVD, format.volume)

    if showDiv and bearDiv
        labelText += "\nDiv: Bear"
    else if showDiv and bullDiv
        labelText += "\nDiv: Bull"

    if showAbs and bearAbsorb
        labelText += "\nAbsorb: Bear"
    else if showAbs and bullAbsorb
        labelText += "\nAbsorb: Bull"

    if showSweep and sweepHigh
        labelText += "\nSweep: Top"

```

```

else if showSweep and sweepLow
    labelText += "\nSweep: Btm"

if showFVG and bearFVG
    labelText += "\nFVG-"
else if showFVG and bullFVG
    labelText += "\nFVG+"

color bgColor = delta > 0.0 ? colorPos : delta < 0.0 ? colorNeg : colorSpread
string tooltipText = "Deep Metrics:\nUp Vol: " + str.toString(safeUp, format.volume) + "\nDn Vol: "
+ str.toString(safeDn, format.volume)

if str.length(labelText) > 0
    label currentLabel = label.new(x = bar_index, y = high, text = labelText, yloc = yloc.abovebar,
color = bgColor, textcolor = colorText, style = label.style_label_down, size = lblSize, tooltip =
tooltipText)

    array.push(activeLabels, currentLabel)
    if array.size(activeLabels) > maxLabels
        label.delete(array.shift(activeLabels))

//
===== //
// UI Engine: Dynamic HTF Analytics Dashboard
//
===== //
var table htfTable = table.new(position.top_right, 6, 5, bgcolor=dashBg, border_width=1,
border_color=dashBorder)

if showDash and barstate.islast
    table.cell(htfTable, 0, 0, "TF", text_color=dashText, text_size=size×small, bgcolor=dashBorder)
    table.cell(htfTable, 1, 0, "Trend", text_color=dashText, text_size=size×small, bgcolor=dashBorder)
    table.cell(htfTable, 2, 0, "Spread", text_color=dashText, text_size=size×small,
bgcolor=dashBorder)
    table.cell(htfTable, 3, 0, "Delta%", text_color=dashText, text_size=size×small,
bgcolor=dashBorder)
    table.cell(htfTable, 4, 0, "FVG", text_color=dashText, text_size=size×small, bgcolor=dashBorder)
    table.cell(htfTable, 5, 0, "Sweep", text_color=dashText, text_size=size×small,

```

```
bgcolor=dashBorder)
```

```
int rowIdx = 1
```

```
if showHtf1
```

```
    color rCol = b1 ? colorPos : colorNeg
```

```
    table.cell(htfTable, 0, rowIdx, htf1, text_color=dashText, text_size=size×small)
```

```
    table.cell(htfTable, 1, rowIdx, b1 ? "UP" : "DN", text_color=rCol, text_size=size×small)
```

```
    table.cell(htfTable, 2, rowIdx, str.toString(s1, format.mintick), text_color=dashText,  
text_size=size×small)
```

```
    table.cell(htfTable, 3, rowIdx, str.toString(d1, "#.##") + "%", text_color=d1 > 0 ? colorPos :  
colorNeg, text_size=size×small)
```

```
    table.cell(htfTable, 4, rowIdx, f1 == 1 ? "+FVG" : f1 == -1 ? "-FVG" : "-", text_color=f1 == 1 ?  
colorPos : f1 == -1 ? colorNeg : dashText, text_size=size×small)
```

```
    table.cell(htfTable, 5, rowIdx, sw1 == 1 ? "Btm" : sw1 == -1 ? "Top" : "-", text_color=sw1 == 1 ?  
colorPos : sw1 == -1 ? colorNeg : dashText, text_size=size×small)
```

```
    rowIdx += 1
```

```
if showHtf2
```

```
    color rCol = b2 ? colorPos : colorNeg
```

```
    table.cell(htfTable, 0, rowIdx, htf2, text_color=dashText, text_size=size×small)
```

```
    table.cell(htfTable, 1, rowIdx, b2 ? "UP" : "DN", text_color=rCol, text_size=size×small)
```

```
    table.cell(htfTable, 2, rowIdx, str.toString(s2, format.mintick), text_color=dashText,  
text_size=size×small)
```

```
    table.cell(htfTable, 3, rowIdx, str.toString(d2, "#.##") + "%", text_color=d2 > 0 ? colorPos :  
colorNeg, text_size=size×small)
```

```
    table.cell(htfTable, 4, rowIdx, f2 == 1 ? "+FVG" : f2 == -1 ? "-FVG" : "-", text_color=f2 == 1 ?  
colorPos : f2 == -1 ? colorNeg : dashText, text_size=size×small)
```

```
    table.cell(htfTable, 5, rowIdx, sw2 == 1 ? "Btm" : sw2 == -1 ? "Top" : "-", text_color=sw2 == 1 ?  
colorPos : sw2 == -1 ? colorNeg : dashText, text_size=size×small)
```

```
    rowIdx += 1
```

```
if showHtf3
```

```
    color rCol = b3 ? colorPos : colorNeg
```

```
    table.cell(htfTable, 0, rowIdx, htf3, text_color=dashText, text_size=size×small)
```

```
    table.cell(htfTable, 1, rowIdx, b3 ? "UP" : "DN", text_color=rCol, text_size=size×small)
```

```
    table.cell(htfTable, 2, rowIdx, str.toString(s3, format.mintick), text_color=dashText,  
text_size=size×small)
```

```

        table.cell(htfTable, 3, rowIdx, str.toString(d3, "#.##") + "%", text_color=d3 > 0 ? colorPos :
colorNeg, text_size=size×small)
        table.cell(htfTable, 4, rowIdx, f3 == 1 ? "+FVG" : f3 == -1 ? "-FVG" : "-", text_color=f3 == 1 ?
colorPos : f3 == -1 ? colorNeg : dashText, text_size=size×small)
        table.cell(htfTable, 5, rowIdx, sw3 == 1 ? "Btm" : sw3 == -1 ? "Top" : "-", text_color=sw3 == 1 ?
colorPos : sw3 == -1 ? colorNeg : dashText, text_size=size×small)
        rowIdx += 1

if showHtf4
    color rCol = b4 ? colorPos : colorNeg
    table.cell(htfTable, 0, rowIdx, htf4, text_color=dashText, text_size=size×small)
    table.cell(htfTable, 1, rowIdx, b4 ? "UP" : "DN", text_color=rCol, text_size=size×small)
    table.cell(htfTable, 2, rowIdx, str.toString(s4, format.mintick), text_color=dashText,
text_size=size×small)
    table.cell(htfTable, 3, rowIdx, str.toString(d4, "#.##") + "%", text_color=d4 > 0 ? colorPos :
colorNeg, text_size=size×small)
    table.cell(htfTable, 4, rowIdx, f4 == 1 ? "+FVG" : f4 == -1 ? "-FVG" : "-", text_color=f4 == 1 ?
colorPos : f4 == -1 ? colorNeg : dashText, text_size=size×small)
    table.cell(htfTable, 5, rowIdx, sw4 == 1 ? "Btm" : sw4 == -1 ? "Top" : "-", text_color=sw4 == 1 ?
colorPos : sw4 == -1 ? colorNeg : dashText, text_size=size.small)

```